Informe de formulación de prompts

Automatización de tareas de integración de datos con inteligencia artificial generativa

Evidencia: AA2-EV01. Informe de formulación de prompts

Aprendiz: Erick Manuel Sang Cifuentes

Instructor: Luis Alejandro Londoño Rivera

Programa: Aplicación de la inteligencia artificial en la integración de datos (código 21720207, ficha 3635322)

Servicio Nacional de Aprendizaje (SENA) · Octubre de 2026

1. Introducción

Este informe documenta el diseño, la prueba y el análisis de cuatro prompts dirigidos a un modelo de inteligencia artificial generativa para automatizar tareas frecuentes de un proceso de integración de datos: transformar una estructura anidada, construir un proceso de extracción, limpieza y combinación de fuentes heterogéneas, redactar documentación técnica y generar un modelo dimensional.

No me limité a pedir código y copiarlo. Cada resultado se **ejecutó sin modificarlo** contra los datos del caso y se **verificó de forma independiente**, de modo que el análisis crítico se apoya en hechos comprobados y no en la apariencia de la respuesta.

Aspecto	Detalle
Herramienta de IA generativa	Claude (Anthropic), usado a través de Claude Code
Fecha de las pruebas	13 de octubre de 2026
Entorno de ejecución	Python 3.12, Polars 2.0.0, DuckDB 1.5.6, Windows 11
Método	Prompt básico (versión 1) → análisis del resultado → prompt refinado (versión 2) → ejecución y verificación
Datos	Simulados y reproducibles; no contienen datos personales reales

2. Caso de estudio y necesidad de integración

Distribuidora Andina (empresa ficticia) necesita analizar sus ventas de 2025, pero la información está repartida en tres fuentes con formatos y problemas de calidad distintos:

Fuente	Formato	Contenido	Problemas de calidad sembrados
`ventas_erp.csv`	CSV, separador `;`, latin-1	1.525 filas de ventas	Dos formatos de fecha, coma decimal, 25 duplicados, llaves con espacios y minúsculas, cantidades nulas y negativas, clientes inexistentes, ciudad escrita de varias formas
`clientes_crm.json`	JSON anidado, UTF-8	120 clientes	Claves anidadas ausentes, correos nulos, segmento sin estandarizar
`productos.csv`	CSV, separador `,`, UTF-8	32 productos	Ninguno

Necesidad concreta: obtener una tabla integrada y confiable, y a partir de ella un esquema en estrella que alimente un tablero en Power BI (evidencia AA2-EV03). El flujo completo es:

```
clientes_crm.json --[Prompt 1: aplanar, Polars]--> clientes_planos.parquet --+
ventas_erp.csv ----+                                                          |
productos.csv -----+--[Prompt 2: ETL, Polars lazy]--> ventas_integradas.parquet
                                                              |
```

```
                    [Prompt 3: documentación] <---+---> [Prompt 4: esquema en estrella, DuckDB]
                                                                  |
                                                  fact_ventas + dim_cliente + dim_producto + dim_fecha
                                                                  |
                                                                Power BI
```

3. Criterios aplicados para formular los prompts

Los prompts refinados aplican los principios y técnicas del componente formativo *Estrategias de integración y análisis de datos mediante inteligencia artificial* (SENA, 2025):

Principio o técnica	Cómo se aplicó
Simulación de roles	"Actúa como ingeniero de datos senior", "redactor técnico", "arquitecto de datos"
Contextualización	Se describe el caso, las fuentes, la codificación y los problemas ya detectados
Uso de ejemplos	Se pegan registros reales de muestra de cada archivo
Claridad	Reglas de negocio numeradas, una decisión por regla
Delimitación del formato	Nombre de la función, columnas exactas y en orden, forma de la respuesta
Control de estilo y extensión	"Máximo 5 viñetas", "máximo 600 palabras", "sin más explicaciones"
Descomposición de tareas	El problema se dividió en cuatro prompts encadenados en lugar de uno solo
Iteración	Cada prompt partió de una versión básica cuyo resultado mostró qué faltaba especificar
Lenguaje no sesgado y veracidad	"No inventes información… escribe *Por confirmar*"

4. Prompt 1 — Transformación de una estructura de datos

Tarea: convertir el JSON anidado del CRM en una tabla plana, lista para cruzarse con las ventas.

4.1 Versión inicial

```
Hazme un script para pasar un JSON de clientes a una tabla.
```

Resultado (resumen). El modelo advirtió que no conocía la estructura y entregó un script genérico de 31 líneas basado en `pandas.json_normalize`. La respuesta completa está en `respuestas/p1_v1.md`.

Qué pasó al ejecutarlo sobre el archivo real:

Aspecto	Resultado con el prompt básico
Nombre de archivo	Asumió `clientes.json`; falla hasta corregirlo a mano
Columnas	`contacto_email`, `direccion_ciudad`…: nombres derivados de la estructura, no los que necesita el cruce
Segmento	Quedaron 4 clientes como `HOGAR` y 58 como `Hogar`: dos categorías para el mismo segmento
Etiquetas	Lista de Python dentro de la celda; no se puede guardar ni filtrar con facilidad
Fecha	Texto, no tipo fecha
Duplicados	Sin validación

El script no estaba "mal": respondía a una pregunta mal formulada. Como yo no di contexto, el modelo tuvo que suponer, y varias suposiciones no coincidieron con mi caso.

4.2 Prompt refinado

````
Actúa como ingeniero de datos senior con experiencia en Python y Polars.

CONTEXTO
Trabajo en la integración de datos de una distribuidora. El CRM exporta los clientes en un archivo
`clientes_crm.json` (UTF-8) con estructura anidada. Necesito aplanarlo a una tabla para poder cruzarlo con las
ventas del ERP. Este es el formato del archivo, con dos registros de ejemplo:

```json
{
  "exportado": "2026-01-05",
  "origen": "CRM",
  "clientes": [
    {
      "id": "C0001",
      "razon_social": "Almacenes Caribe 1",
      "segmento": "Pyme",
      "contacto": {"email": "compras1@ejemplo.com.co", "telefono": "6064514209"},
      "direccion": {"ciudad": "Pereira", "departamento": "Risaralda"},
      "fecha_registro": "2023-05-22",
      "etiquetas": ["nuevo", "web"]
    },
    {
      "id": "C0023",
      "razon_social": "Grupo Central 23",
      "segmento": "HOGAR",
      "contacto": {"email": null, "telefono": "6013456789"},
      "direccion": {"ciudad": "Bogotá"},
      "fecha_registro": "2022-11-03",
      "etiquetas": []
    }
  ]
}
```

Problemas que ya identifiqué: algunas claves anidadas no existen (por ejemplo `departamento`), hay correos en
null y el segmento no está estandarizado (`Hogar` / `HOGAR`).

TAREA
Escribe una función `aplanar_clientes(ruta_json) -> polars.DataFrame` que devuelva una fila por cliente con
exactamente estas columnas y en este orden:
`cod_cliente, razon_social, segmento, email, telefono, ciudad, departamento, fecha_registro, etiquetas`

REGLAS
1. `cod_cliente` sale de `id`, en mayúsculas y sin espacios.
2. `segmento` con la primera letra en mayúscula y el resto en minúscula (Hogar, Pyme, Corporativo).
3. Las claves ausentes o en null quedan como valor nulo; el script no debe fallar por ellas.
````

```
4. `fecha_registro` como tipo `pl.Date`.
5. `etiquetas` como texto único separado por `|` (lista vacía -> cadena vacía).
6. `telefono` se conserva como texto.
7. Si hay `cod_cliente` repetidos, lanza un `ValueError` indicando cuáles.

FORMATO DE SALIDA
- Un único bloque de código Python (solo Polars y la biblioteca estándar; tengo instalado Polars 2.0, usa
únicamente API vigente desde la 1.x, sin funciones obsoletas), con docstring y comentarios breves en español. Usa
expresiones de Polars; no uses `map_elements` ni bucles fila a fila sobre el DataFrame.
- Un bloque `if __name__ == "__main__":` que lea `clientes_crm.json`, imprima número de filas, nulos por columna
y guarde `clientes_planos.parquet`.
- Después del código, máximo 5 viñetas con los supuestos que tomaste. No agregues explicaciones adicionales.
```

4.3 Resultado generado por la IA (literal)

```
"""Aplana el export de clientes del CRM (JSON anidado) a una tabla de Polars."""

import json
from pathlib import Path

import polars as pl

# Esquema explícito: las claves ausentes en el JSON quedan como nulos
# y las claves no declaradas se descartan.
ESQUEMA_CRM = {
    "id": pl.String,
    "razon_social": pl.String,
    "segmento": pl.String,
    "contacto": pl.Struct({"email": pl.String, "telefono": pl.String}),
    "direccion": pl.Struct({"ciudad": pl.String, "departamento": pl.String}),
    "fecha_registro": pl.String,
    "etiquetas": pl.List(pl.String),
}

COLUMNAS_SALIDA = [
    "cod_cliente",
    "razon_social",
    "segmento",
    "email",
    "telefono",
    "ciudad",
    "departamento",
    "fecha_registro",
    "etiquetas",
]


def aplanar_clientes(ruta_json) -> pl.DataFrame:
    """Devuelve una fila por cliente a partir del export anidado del CRM.

    Parámetros
    ----------
    ruta_json : str | pathlib.Path
        Ruta del archivo JSON (UTF-8) con la clave raíz ``clientes``.

    Retorna
    -------
    polars.DataFrame
        Columnas, en este orden: cod_cliente, razon_social, segmento, email,
        telefono, ciudad, departamento, fecha_registro (pl.Date), etiquetas.

    Lanza
    -----
    ValueError
        Si la clave ``clientes`` no es una lista o si hay ``cod_cliente``
        repetidos (el mensaje indica cuáles).
    """
    with Path(ruta_json).open(encoding="utf-8") as archivo:
        contenido = json.load(archivo)
```

```
    clientes = contenido.get("clientes")
    if not isinstance(clientes, list):
        raise ValueError("El JSON no contiene una lista en la clave 'clientes'.")

    # Carga con esquema fijo: tolera claves ausentes y valores null.
    crudo = pl.DataFrame(clientes, schema=ESQUEMA_CRM)

    segmento = pl.col("segmento").str.strip_chars()

    df = crudo.select(
        # Regla 1: mayúsculas y sin espacios (internos ni externos).
        pl.col("id").str.replace_all(r"\s+", "").str.to_uppercase().alias("cod_cliente"),
        pl.col("razon_social"),
        # Regla 2: primera letra en mayúscula y el resto en minúscula.
        pl.concat_str(
            segmento.str.slice(0, 1).str.to_uppercase(),
            segmento.str.slice(1).str.to_lowercase(),
        ).alias("segmento"),
        # Regla 3: un struct nulo o un campo ausente produce nulo, sin error.
        pl.col("contacto").struct.field("email").alias("email"),
        # Regla 6: el teléfono se conserva como texto.
        pl.col("contacto").struct.field("telefono").alias("telefono"),
        pl.col("direccion").struct.field("ciudad").alias("ciudad"),
        pl.col("direccion").struct.field("departamento").alias("departamento"),
        # Regla 4: texto ISO a pl.Date.
        pl.col("fecha_registro").str.to_date("%Y-%m-%d").alias("fecha_registro"),
        # Regla 5: lista a texto separado por "|"; lista vacía -> "".
        pl.col("etiquetas").list.join("|", ignore_nulls=True).alias("etiquetas"),
    ).select(COLUMNAS_SALIDA)

    # Regla 7: códigos repetidos (se evalúa después de normalizar).
    repetidos = (
        df.filter(
            pl.col("cod_cliente").is_not_null() & pl.col("cod_cliente").is_duplicated()
        )
        .get_column("cod_cliente")
        .unique()
        .sort()
        .to_list()
    )
    if repetidos:
        raise ValueError(f"cod_cliente repetidos: {', '.join(repetidos)}")

    return df


if __name__ == "__main__":
    clientes = aplanar_clientes("clientes_crm.json")

    print(f"Filas: {clientes.height}")
    print("Nulos por columna:")
    print(clientes.null_count())

    clientes.write_parquet("clientes_planos.parquet")
```

- No puedo confirmar la existencia de Polars 2.0 ni sus cambios; el código usa solo API estable de la serie 1.x (`struct.field`, `str.to_date`, `list.join`, `concat_str`), sin funciones marcadas como obsoletas.
- Los duplicados se evalúan sobre `cod_cliente` ya normalizado, así que `c0001` y `C0001` cuentan como repetidos; los `id` nulos no se tratan como duplicados y quedan como nulo.
- `fecha_registro` llega siempre como `AAAA-MM-DD`; una fecha con otro formato detiene la ejecución con error en lugar de convertirse en nulo en silencio.
- Si `etiquetas` no existe o es null queda nulo (regla 3); solo la lista vacía se convierte en cadena vacía, y las etiquetas nulas dentro de una lista se omiten.
- Los tipos del JSON son los del ejemplo (`telefono` e `id` como texto); un tipo distinto hace fallar la carga. A `segmento` también se le quitan los espacios de los extremos.

4.4 Ejecución real

```
Filas: 120
Nulos por columna:
shape: (1, 9)
┌──────────────┬──────────────┬──────────┬─────────┬─────┬─────────┬──────────────┬──────────────┬──────────┐
│ cod_cliente  ┆ razon_socia  ┆ segmento ┆ email   ┆ …   ┆ ciudad  ┆ departament  ┆ fecha_regi   ┆ etiquetas │
│ ---          ┆ l            ┆ ---      ┆ ---     ┆     ┆ ---     ┆ o            ┆ stro         ┆ ---      │
│ u32          ┆ ---          ┆ u32      ┆ u32     ┆     ┆ u32     ┆ ---          ┆ ---          ┆ u32      │
│              ┆ u32          ┆          ┆         ┆     ┆         ┆ u32          ┆ u32          ┆          │
╞══════════════╪══════════════╪══════════╪═════════╪═════╪═════════╪══════════════╪══════════════╪══════════╡
│ 0            ┆ 0            ┆ 0        ┆ 7       ┆ …   ┆ 0       ┆ 5            ┆ 0            ┆ 0        │
└──────────────┴──────────────┴──────────┴─────────┴─────┴─────────┴──────────────┴──────────────┴──────────┘
```

4.5 Análisis crítico

- **¿Fue útil?** Sí. El código corrió al primer intento sobre Polars 2.0.0, sin ajustes, y produjo las 120 filas con las 9 columnas en el orden pedido. Los 3 segmentos quedaron estandarizados (62 Hogar, 37 Pyme, 21 Corporativo) y los nulos coinciden con los defectos sembrados: 7 correos y 5 departamentos.
- **Verificación independiente.** Leí el Parquet resultante y comprobé tipos (`fecha_registro` como `Date`, `telefono` como texto) y que las 28 listas vacías de etiquetas quedaron como cadena vacía y no como nulo.
- **Ajustes realizados.** Ninguno al código. El ajuste estuvo en el prompt: pasar de una frase a un encargo con contexto, ejemplo, reglas y formato.
- **Limitación detectada.** El modelo declaró que no podía confirmar la existencia de Polars 2.0 y que usó solo API estable de la serie 1.x. Es una advertencia honesta y un riesgo real: la única forma de cerrarlo fue ejecutar el código. Un resultado de IA sobre una versión que el modelo no conoce no debe usarse sin prueba.
- **Impacto en la tarea.** Con el prompt básico habría tenido que reescribir casi todo; con el refinado, la tarea quedó resuelta en una sola interacción.

---

5. Prompt 2 — Extracción, limpieza y combinación de fuentes heterogéneas

Tarea: generar el proceso ETL que integra ventas, clientes y productos en una sola tabla analítica.

5.1 Versión inicial

```
Necesito un script en Python que limpie un archivo de ventas y lo una con los clientes y los productos.
```

Resultado (resumen). Un script de 133 líneas en pandas, bien organizado y con un bloque de configuración, pero construido sobre supuestos: archivos `ventas.csv`, `clientes.csv` y `productos.csv`, columnas `id_cliente` e `id_producto`, separador `,` y codificación UTF-8. La respuesta completa está en `respuestas/p2_v1.md`.

Qué pasó al ejecutarlo sobre los archivos reales:

Prueba	Resultado
Tal cual, cambiando solo los nombres de archivo	Falla: `UnicodeDecodeError` (el archivo está en latin-1, no en UTF-8)
Ajustando a mano separador, codificación y nombres de columnas	**0 filas** después de la limpieza: descartó las 1.525

El segundo resultado es el más instructivo. El script no falla: termina "bien" e informa que descartó todo. La causa es que el precio viene con coma decimal (`55455,49`); al convertirlo a número queda nulo y la regla "descartar filas sin precio" elimina el 100 % de los datos. Además, su regla `cantidad > 0` habría eliminado las devoluciones, una decisión de negocio que el modelo tomó por mí porque yo no la especifiqué.

5.2 Prompt refinado

````
Actúa como ingeniero de datos senior. Necesito un script ETL en Python con Polars (API lazy) que extraiga, limpie
y combine tres fuentes heterogéneas en una sola tabla analítica.

FUENTES
1) `ventas_erp.csv` — exportación del ERP. Separador `;`, codificación latin-1. Muestra:
```
id_venta;fecha;cod_cliente;sku;cantidad;precio_unitario;canal;ciudad
V00296;28/03/2025;C0119;SKU-026;10;55455,49;Tienda;Bogotá
V00232;2025-03-11; c0019 ;sku-015;2;154166,14;Tienda;BOGOTA
V00779;04/08/2025;C0098;SKU-014;;235614,32;Web;Medellín
```
2) `clientes_planos.parquet` — clientes del CRM ya aplanados. Columnas: `cod_cliente, razon_social, segmento,
email, telefono, ciudad, departamento, fecha_registro, etiquetas` (todas texto salvo `fecha_registro`, que es
fecha).
3) `productos.csv` — catálogo. Separador `,`, UTF-8. Columnas: `sku, nombre_producto, categoria, precio_lista,
costo_unitario`.

PROBLEMAS DE CALIDAD YA DETECTADOS EN `ventas_erp.csv`
- Fechas en dos formatos: `dd/mm/aaaa` y `aaaa-mm-dd`.
- `precio_unitario` con coma decimal.
- Filas completamente duplicadas.
- `cod_cliente` y `sku` con espacios y minúsculas.
- `cantidad` nula en algunas filas y negativa en otras.
- `precio_unitario` nulo en unas pocas filas.
- Códigos de cliente que no existen en el CRM.
- La columna `ciudad` del ERP está escrita de muchas formas (BOGOTA, Bogota, " bogota").

REGLAS DE NEGOCIO
1. Eliminar duplicados exactos.
2. Normalizar llaves: `cod_cliente` y `sku` en mayúsculas y sin espacios.
3. Convertir `fecha` a tipo fecha aceptando ambos formatos; si una fecha no se puede interpretar, la fila se
descarta y se cuenta.
4. Filas con `cantidad` nula o `precio_unitario` nulo: se descartan y se cuentan (no imputar).
5. `cantidad` negativa: NO se descarta; se conserva y se marca con `es_devolucion = True`.
6. La ciudad y el departamento se toman del CRM, no del ERP (descartar la columna `ciudad` del ERP).
7. Unir con LEFT JOIN desde ventas. Las ventas cuyo cliente no existe en el CRM se conservan y se marcan con
`cliente_sin_crm = True`.
8. Calcular `total_venta = cantidad * precio_unitario` y `margen = (precio_unitario - costo_unitario) *
cantidad`.

SALIDA ESPERADA
Archivo `ventas_integradas.parquet` con estas columnas en este orden (tipos: `fecha` Date, `cantidad` Int64,
importes Float64, banderas Boolean):
`id_venta, fecha, cod_cliente, razon_social, segmento, ciudad, departamento, sku, nombre_producto, categoria,
canal, cantidad, precio_unitario, costo_unitario, total_venta, margen, es_devolucion, cliente_sin_crm`

FORMATO DE LA RESPUESTA
- Un único script organizado en funciones `extraer()`, `limpiar_ventas()`, `integrar()` y `main()`, con
comentarios breves en español.
- Al final debe imprimir un reporte de calidad: filas leídas, duplicados eliminados, filas descartadas por cada
````

```
regla, devoluciones marcadas, ventas sin cliente en CRM y filas finales.
- Incluye validaciones con `assert` para: unicidad de `id_venta` en la salida y que el número de filas no aumente después de los joins.
- Solo Polars y biblioteca estándar. Tengo instalado Polars 2.0: usa únicamente API vigente desde la 1.x, sin funciones obsoletas. Lee con `scan_csv` / `scan_parquet` y ejecuta con `collect()` solo cuando haga falta; no uses `map_elements`.
- Después del código, una lista corta (máximo 5 viñetas) de supuestos o riesgos. Sin más explicaciones.
```

5.3 Resultado generado por la IA (literal)

```python
"""ETL de ventas: ERP (CSV) + CRM (Parquet) + catálogo (CSV) -> ventas_integradas.parquet"""

import shutil
import tempfile
from pathlib import Path

import polars as pl

RUTA_VENTAS = Path("ventas_erp.csv")
RUTA_CLIENTES = Path("clientes_planos.parquet")
RUTA_PRODUCTOS = Path("productos.csv")
RUTA_SALIDA = Path("ventas_integradas.parquet")

# Columnas de salida, en orden, con su tipo final
ESQUEMA_SALIDA = {
    "id_venta": pl.String,
    "fecha": pl.Date,
    "cod_cliente": pl.String,
    "razon_social": pl.String,
    "segmento": pl.String,
    "ciudad": pl.String,
    "departamento": pl.String,
    "sku": pl.String,
    "nombre_producto": pl.String,
    "categoria": pl.String,
    "canal": pl.String,
    "cantidad": pl.Int64,
    "precio_unitario": pl.Float64,
    "costo_unitario": pl.Float64,
    "total_venta": pl.Float64,
    "margen": pl.Float64,
    "es_devolucion": pl.Boolean,
    "cliente_sin_crm": pl.Boolean,
}


def _llave(columna: str) -> pl.Expr:
    """Normaliza una llave: sin espacios (tampoco internos) y en mayúsculas."""
    return pl.col(columna).str.replace_all(r"\s+", "").str.to_uppercase()


def extraer(dir_tmp: Path) -> tuple[pl.LazyFrame, pl.LazyFrame, pl.LazyFrame]:
    """Abre las tres fuentes en modo lazy, todo como texto en los CSV."""
    # scan_csv solo acepta UTF-8: se transcodifica el latin-1 a una copia temporal
    ruta_utf8 = dir_tmp / "ventas_erp_utf8.csv"
    with (
        open(RUTA_VENTAS, encoding="latin-1", newline="") as origen,
        open(ruta_utf8, "w", encoding="utf-8", newline="") as destino,
    ):
        shutil.copyfileobj(origen, destino)

    ventas = pl.scan_csv(ruta_utf8, separator=";", infer_schema=False)
    clientes = pl.scan_parquet(RUTA_CLIENTES)
    productos = pl.scan_csv(RUTA_PRODUCTOS, separator=",", infer_schema=False)
    return ventas, clientes, productos


def limpiar_ventas(ventas: pl.LazyFrame) -> tuple[pl.LazyFrame, dict[str, int]]:
    """Aplica las reglas 1 a 5 y devuelve las ventas limpias junto con los conteos."""
    fecha_txt = pl.col("fecha").str.strip_chars()
```

```
    marcadas = (
        ventas
        # Regla 1: se marca el duplicado exacto sobre la fila cruda, antes de transformar
        .with_columns(_duplicada=~pl.struct(pl.all()).is_first_distinct())
        .with_columns(
            pl.col("id_venta").str.strip_chars(),
            # Regla 3: se prueban ambos formatos; si ninguno aplica queda nulo
            pl.coalesce(
                fecha_txt.str.to_date("%d/%m/%Y", strict=False),
                fecha_txt.str.to_date("%Y-%m-%d", strict=False),
            ).alias("fecha"),
            # Regla 2: llaves normalizadas
            _llave("cod_cliente"),
            _llave("sku"),
            pl.col("cantidad").str.strip_chars().cast(pl.Int64, strict=False),
            # Coma decimal -> punto decimal
            pl.col("precio_unitario")
            .str.strip_chars()
            .str.replace(",", ".", literal=True)
            .cast(pl.Float64, strict=False),
            pl.col("canal").str.strip_chars(),
        )
        # Regla 6: la ciudad del ERP no se usa
        .drop("ciudad")
        # collect necesario: los conteos de calidad requieren materializar las ventas
        .collect()
    )

    unica = ~pl.col("_duplicada")
    fecha_mala = pl.col("fecha").is_null()
    cantidad_nula = pl.col("cantidad").is_null()
    precio_nulo = pl.col("precio_unitario").is_null()

    # Los descartes se atribuyen en cascada para que el reporte cuadre
    conteos = marcadas.select(
        filas_leidas=pl.len(),
        duplicados=pl.col("_duplicada").sum(),
        desc_fecha=(unica & fecha_mala).sum(),
        desc_cantidad=(unica & ~fecha_mala & cantidad_nula).sum(),
        desc_precio=(unica & ~fecha_mala & ~cantidad_nula & precio_nulo).sum(),
    ).row(0, named=True)

    limpias = (
        marcadas
        # Reglas 1, 3 y 4: se descartan duplicados, fechas inválidas y nulos
        .filter(unica & ~fecha_mala & ~cantidad_nula & ~precio_nulo)
        .drop("_duplicada")
        # Regla 5: la cantidad negativa se conserva y se marca
        .with_columns(es_devolucion=pl.col("cantidad") < 0)
    )
    conteos["filas_limpias"] = limpias.height
    return limpias.lazy(), conteos


def integrar(
    ventas: pl.LazyFrame, clientes: pl.LazyFrame, productos: pl.LazyFrame
) -> pl.LazyFrame:
    """Aplica las reglas 6 a 8: joins con CRM y catálogo, y cálculo de importes."""
    dim_clientes = clientes.select(
        _llave("cod_cliente"),
        "razon_social",
        "segmento",
        "ciudad",
        "departamento",
        pl.lit(True).alias("_en_crm"),  # testigo para detectar clientes sin CRM
    )
    dim_productos = productos.select(
        _llave("sku"),
        "nombre_producto",
        "categoria",
        pl.col("costo_unitario").str.strip_chars().cast(pl.Float64, strict=False),
    )
```

```
    return (
        ventas
        # Regla 7: LEFT JOIN desde ventas
        .join(dim_clientes, on="cod_cliente", how="left")
        .join(dim_productos, on="sku", how="left")
        .with_columns(
            cliente_sin_crm=pl.col("_en_crm").is_null(),
            # Regla 8: importes
            total_venta=pl.col("cantidad") * pl.col("precio_unitario"),
            margen=(pl.col("precio_unitario") - pl.col("costo_unitario"))
            * pl.col("cantidad"),
        )
        .select(pl.col(nombre).cast(tipo) for nombre, tipo in ESQUEMA_SALIDA.items())
        .sort("fecha", "id_venta")
    )


def main() -> None:
    with tempfile.TemporaryDirectory(ignore_cleanup_errors=True) as tmp:
        ventas, clientes, productos = extraer(Path(tmp))
        ventas_limpias, conteos = limpiar_ventas(ventas)

    resultado = integrar(ventas_limpias, clientes, productos).collect()

    # Validaciones antes de escribir
    assert resultado.height <= conteos["filas_limpias"], (
        "Los joins aumentaron las filas: hay llaves repetidas en clientes o productos"
    )
    assert resultado["id_venta"].is_unique().all(), "id_venta no es único en la salida"

    resultado.write_parquet(RUTA_SALIDA)

    # Reporte de calidad
    print("REPORTE DE CALIDAD")
    print(f"Filas leídas:                        {conteos['filas_leidas']}")
    print(f"Duplicados eliminados:               {conteos['duplicados']}")
    print(f"Descartadas por fecha inválida:      {conteos['desc_fecha']}")
    print(f"Descartadas por cantidad nula:       {conteos['desc_cantidad']}")
    print(f"Descartadas por precio nulo:         {conteos['desc_precio']}")
    print(f"Devoluciones marcadas:               {resultado['es_devolucion'].sum()}")
    print(f"Ventas sin cliente en CRM:           {resultado['cliente_sin_crm'].sum()}")
    print(f"Ventas sin costo (SKU sin catálogo): {resultado['costo_unitario'].null_count()}")
    print(f"Filas finales:                       {resultado.height}")
    print(f"Archivo generado:                    {RUTA_SALIDA}")


if __name__ == "__main__":
    main()
```

Supuestos y riesgos

- **Sin ejecutar y sin verificar contra Polars 2.0:** no ejecuté el script y no conozco esa versión. Solo uso API estable de la 1.x, sin `map_elements`; requiere Python 3.10 o superior.
- **Latin-1:** `scan_csv` solo lee UTF-8, así que el script crea una copia temporal transcodificada de `ventas_erp.csv` y la borra al terminar.
- **Llaves repetidas:** los duplicados se evalúan sobre la fila cruda. Si un `id_venta` se repite con algún campo distinto, o si el CRM o el catálogo traen llaves repetidas, el script no deduplica y se detiene en los `assert`.
- **Valores no convertibles:** una cantidad como `2.5` o un precio con separador de miles (`1.234,56`) quedan nulos y se cuentan como descarte por nulo. Los descartes se atribuyen en cascada (fecha, luego cantidad, luego precio), así que cada fila cuenta una sola vez.
- **SKU sin catálogo:** la venta se conserva con `costo_unitario` y `margen` nulos, y el reporte la cuenta en una línea adicional. Las devoluciones producen `total_venta` y `margen` negativos.

5.4 Ejecución real

```
REPORTE DE CALIDAD
Filas leídas:                        1525
Duplicados eliminados:               25
Descartadas por fecha inválida:      0
Descartadas por cantidad nula:       30
Descartadas por precio nulo:         5
Devoluciones marcadas:               12
Ventas sin cliente en CRM:           8
Ventas sin costo (SKU sin catálogo): 0
Filas finales:                       1465
Archivo generado:                    ventas_integradas.parquet
```

5.5 Análisis crítico

- **¿Fue útil?** Sí. Corrió al primer intento y el reporte de calidad coincide exactamente con los defectos sembrados en los datos: 25 duplicados, 30 cantidades nulas, 5 precios nulos, 12 devoluciones y 8 ventas sin cliente en el CRM. La conciliación cuadra: 1.525 − 25 − 30 − 5 = 1.465.
- **Verificación independiente.** Recalculé con pandas, desde el CSV crudo y con otra lógica: las 1.465 fechas coinciden una a una (ninguna con día y mes invertidos) y el total vendido es idéntico, $730.737.516. Dos implementaciones distintas que llegan al mismo número dan mucha más confianza que revisar el código a ojo.
- **Un acierto que yo no había previsto.** Mi prompt pedía leer con `scan_csv` un archivo en latin-1, y esas dos condiciones chocan: la lectura *lazy* de Polars solo admite UTF-8. El modelo lo detectó, lo resolvió con una copia temporal transcodificada y lo declaró en los supuestos. Me mostró una contradicción de mi propio requerimiento.
- **Ajustes pendientes.** (1) `departamento` queda nulo en 67 filas: 8 por clientes sin CRM y 59 por clientes cuyo registro no trae departamento; como la ciudad sí existe, se puede completar con una tabla ciudad → departamento. (2) Un precio con separador de miles (`1.234,56`) quedaría nulo y la fila se descartaría; el script lo advierte, pero conviene manejarlo si la fuente cambia.
- **Impacto en la tarea.** El prompt básico produjo un resultado silenciosamente equivocado; el refinado, un proceso verificable, con conteos por regla y validaciones que detienen la ejecución si algo no cuadra.

---

6. Prompt 3 — Redacción de documentación técnica

Tarea: documentar la tabla integrada para analistas que no participaron en su construcción.

6.1 Versión inicial

```
Documenta la tabla de ventas integradas.
```

6.2 Resultado de la versión inicial (literal)

> No puedo documentar la tabla todavía porque no la tengo: en tu mensaje no viene su estructura ni ningún dato, y si la describiera ahora estaría inventando columnas y reglas que quizá no existen.
>
> Para documentarla necesito que me pegues, como mínimo, una de estas dos cosas:

> - **La estructura de la tabla**: el `CREATE TABLE`, o la lista de columnas con su tipo de dato.
> - **Una muestra de los datos**: el encabezado y entre 5 y 10 filas.
>
> Y si lo tienes a mano, esto mejora mucho el resultado:
>
> 1. **Origen**: de qué fuentes se integró (sistemas, archivos, tablas) y cómo se cruzan entre sí.
> 2. **Granularidad**: qué representa una fila (una venta, una línea de factura, un total diario por tienda).
> 3. **Claves**: la llave primaria y las columnas que enlazan con otras tablas.
> 4. **Transformaciones**: limpiezas, conversiones de moneda o fecha, eliminación de duplicados y columnas calculadas.
> 5. **Audiencia**: si la documentación es para un equipo técnico, para usuarios de negocio o para una entrega académica, y si debe seguir algún formato.
>
> Con eso te entrego la descripción general de la tabla, el diccionario de datos columna por columna (nombre, tipo, descripción, valores permitidos, si admite nulos, origen), las reglas de calidad y las observaciones de uso.
>
> Si lo que buscas es una plantilla genérica para rellenarla tú, dímelo y te la preparo.

El modelo **no inventó** una documentación: pidió la estructura, una muestra y el contexto. Es el comportamiento correcto frente a un prompt sin información, y a la vez la mejor demostración de que el problema estaba en el prompt. De hecho, su lista de lo que necesitaba me sirvió de guía para la versión 2.

6.3 Prompt refinado

```
Actúa como redactor técnico especializado en gobierno de datos. Redacta la documentación técnica de la tabla
`ventas_integradas`, que es el resultado de un proceso ETL en Python con Polars y se almacena como
`ventas_integradas.parquet`.

AUDIENCIA
Analistas de datos que usarán la tabla en Power BI y Python, y que no participaron en su construcción.

CONTEXTO DEL PROCESO
- Fuentes: `ventas_erp.csv` (ERP; separador `;`, latin-1), `clientes_crm.json` (CRM; JSON anidado que se aplana
previamente a `clientes_planos.parquet`) y `productos.csv` (catálogo).
- Periodo de los datos: 1 de enero a 31 de diciembre de 2025. Moneda: pesos colombianos (COP).
- Reglas de transformación aplicadas:
  1. Se eliminan filas duplicadas exactas.
  2. `cod_cliente` y `sku` se normalizan a mayúsculas sin espacios.
  3. `fecha` se convierte a tipo fecha desde dos formatos (`dd/mm/aaaa` y `aaaa-mm-dd`).
  4. Se descartan filas con `cantidad` o `precio_unitario` nulos (no se imputan).
  5. Las cantidades negativas se conservan y se marcan con `es_devolucion = True`.
  6. Ciudad y departamento provienen del CRM; la ciudad del ERP se descarta.
  7. LEFT JOIN desde ventas; las ventas sin cliente en el CRM se marcan con `cliente_sin_crm = True`.
  8. `total_venta = cantidad * precio_unitario`; `margen = (precio_unitario - costo_unitario) * cantidad`.
- Resultado de la última ejecución: 1.525 filas leídas, 25 duplicados eliminados, 30 filas descartadas por
cantidad nula, 5 por precio nulo, 0 por fecha no interpretable, 12 devoluciones marcadas, 8 ventas sin cliente en
CRM, 1.465 filas finales.
- Nulos en la salida: `razon_social`, `segmento` y `ciudad` en 8 filas (las ventas sin cliente en CRM);
`departamento` en 67 filas (esas 8 más las de clientes cuyo registro en el CRM no trae departamento).

COLUMNAS DE LA TABLA (en orden)
`id_venta, fecha, cod_cliente, razon_social, segmento, ciudad, departamento, sku, nombre_producto, categoria,
canal, cantidad, precio_unitario, costo_unitario, total_venta, margen, es_devolucion, cliente_sin_crm`

Valores conocidos: `segmento` ∈ {Hogar, Pyme, Corporativo}; `canal` ∈ {Tienda, Web, Televentas}; `categoria` ∈
{Tecnologia, Hogar, Oficina, Deportes}. Formato de llaves: `id_venta` como `V00296`, `cod_cliente` como `C0119`,
`sku` como `SKU-026`.

ENTREGABLE (en Markdown, en español, tono técnico y conciso, máximo 600 palabras sin contar tablas)
1. **Descripción general**: propósito, granularidad (qué representa una fila) y llave primaria.
2. **Diccionario de datos**: tabla con columnas `Campo | Tipo | Descripción | Fuente de origen | Regla de
transformación | Ejemplo`.
3. **Linaje**: diagrama en texto (fuente -> transformación -> destino).
4. **Reglas de calidad y resultados de la última ejecución**: tabla.
```

```
5. **Limitaciones y advertencias de uso**: en particular cómo tratar devoluciones y clientes sin CRM al calcular
indicadores.

RESTRICCIONES
- No inventes información que no esté en este mensaje. Si falta un dato necesario (por ejemplo, responsable de la
tabla o frecuencia de actualización), escribe "Por confirmar".
- Los ejemplos del diccionario deben ser coherentes con el tipo y los valores conocidos.
```

6.4 Resultado generado por la IA (literal)

> ## Documentación técnica: `ventas_integradas`
>
> ### 1. Descripción general
>
> **Propósito.** Tabla analítica que integra las ventas del ERP con los atributos de cliente del CRM y los del catálogo de productos, lista para consumo en Power BI y Python. Se almacena como `ventas_integradas.parquet` y se genera con un proceso ETL en Python con Polars.
>
> **Cobertura.** 1 de enero a 31 de diciembre de 2025. Valores monetarios en pesos colombianos (COP).
>
> **Granularidad.** Una fila representa un registro de venta del ERP (una venta o una devolución), identificado por `id_venta`.
>
> **Llave primaria.** `id_venta`. La unicidad no está validada por una regla explícita del proceso: Por confirmar.
>
> **Responsable de la tabla:** Por confirmar. **Frecuencia de actualización:** Por confirmar.
>
> ### 2. Diccionario de datos
>
> Los tipos son lógicos; el tipo físico exacto en el Parquet (por ejemplo, entero o decimal) está por confirmar. Donde el mensaje de origen no indica la fuente de un campo, se marca "(por confirmar)".

Campo	Tipo	Descripción	Fuente de origen	Regla de transformación	Ejemplo
`id_venta`	Texto	Identificador del registro de venta	`ventas_erp.csv`	Ninguna	`V00296`
`fecha`	Fecha	Fecha de la venta	`ventas_erp.csv`	Conversión a fecha desde `dd/mm/aaaa` y `aaaa-mm-dd`	`2025-03-14`
`cod_cliente`	Texto	Código del cliente	`ventas_erp.csv`	Mayúsculas, sin espacios	`C0119`
`razon_social`	Texto	Nombre o razón social del cliente	`clientes_crm.json`	LEFT JOIN por `cod_cliente`; nulo si no hay cliente en CRM	`Comercial Andina S.A.S.`
`segmento`	Texto	Segmento del cliente: Hogar, Pyme o Corporativo	`clientes_crm.json`	LEFT JOIN; nulo si no hay cliente en CRM	`Pyme`
`ciudad`	Texto	Ciudad del cliente	`clientes_crm.json`	Se toma del CRM; la ciudad del ERP se descarta	`Medellín`
`departamento`	Texto	Departamento del cliente	`clientes_crm.json`	Se toma del CRM; nulo si el CRM no lo trae	`Antioquia`
`sku`	Texto	Código del producto	`ventas_erp.csv`	Mayúsculas, sin espacios	`SKU-026`
`nombre_producto`	Texto	Nombre del producto	`productos.csv` (por confirmar)	Cruce por `sku`	`Silla ergonómica`
`categoria`	Texto	Categoría: Tecnologia, Hogar, Oficina o Deportes	`productos.csv` (por confirmar)	Cruce por `sku`	`Oficina`
`canal`	Texto	Canal de venta: Tienda, Web o Televentas	`ventas_erp.csv` (por confirmar)	Ninguna	`Web`
`cantidad`	Numérico	Unidades vendidas; negativa en devoluciones	`ventas_erp.csv`	Se descartan filas con nulo; los negativos se conservan	`3`
`precio_unitario`	Numérico (COP)	Precio de venta por unidad	`ventas_erp.csv`	Se descartan filas con nulo	`185000`
`costo_unitario`	Numérico (COP)	Costo por unidad	`productos.csv` (por confirmar)	Cruce por `sku`	`120000`
`total_venta`	Numérico (COP)	Valor total del registro	Calculado	`cantidad * precio_unitario`	`555000`
`margen`	Numérico (COP)	Margen bruto del registro	Calculado	`(precio_unitario - costo_unitario) * cantidad`	`195000`

Campo	Tipo	Descripción	Fuente de origen	Regla de transformación	Ejemplo
`es_devolucion`	Booleano	Indica que el registro es una devolución	Calculado	`True` si `cantidad` es negativa	`False`
`cliente_sin_crm`	Booleano	Indica que la venta no tiene cliente en el CRM	Calculado	`True` si el LEFT JOIN no encuentra el cliente	`False`

3. Linaje

```
clientes_crm.json (CRM, JSON anidado)
  -> aplanado
  -> clientes_planos.parquet ------------------------------+
                                                           |
ventas_erp.csv (ERP; separador ";", latin-1)               |
  -> eliminar duplicados exactos                           |
  -> normalizar cod_cliente y sku (mayúsculas, sin espacios)
  -> convertir fecha (dd/mm/aaaa, aaaa-mm-dd)              |
  -> descartar cantidad o precio_unitario nulos            |
  -> marcar es_devolucion (cantidad negativa)              |
  -> descartar ciudad del ERP                              |
  -> LEFT JOIN por cod_cliente <---------------------------+
       -> marcar cliente_sin_crm
  -> cruce por sku <--- productos.csv (catálogo)
  -> calcular total_venta y margen
  -> ventas_integradas.parquet
```

El orden exacto de los pasos y el tipo de cruce con el catálogo están por confirmar.

4. Reglas de calidad y resultados de la última ejecución

Fecha de la última ejecución: Por confirmar.

Regla	Acción	Resultado
Filas leídas del ERP	—	1.525
Duplicados exactos	Se eliminan	25
`cantidad` nula	Se descarta la fila (no se imputa)	30
`precio_unitario` nulo	Se descarta la fila (no se imputa)	5
`fecha` no interpretable	—	0
`cantidad` negativa	Se conserva, `es_devolucion = True`	12
Venta sin cliente en CRM	Se conserva, `cliente_sin_crm = True`	8
Filas finales	—	**1.465**

Conciliación: 1.525 − 25 − 30 − 5 = 1.465.

Campo con nulos en la salida	Filas	Causa
`razon_social`, `segmento`, `ciudad`	8	Ventas sin cliente en CRM
`departamento`	67	Las 8 anteriores más 59 de clientes cuyo registro en el CRM no trae departamento

> ### 5. Limitaciones y advertencias de uso
>
> **Devoluciones (12 filas).** Tienen `cantidad`, `total_venta` y `margen` negativos. - Una suma directa de `total_venta` o `margen` da el valor **neto** de devoluciones. - Para ventas **brutas**, filtre `es_devolucion = False`. - Exclúyalas al contar transacciones y al calcular ticket promedio o unidades por venta; si no, distorsionan el resultado. - La tabla no enlaza la devolución con su venta original.
>
> **Clientes sin CRM (8 filas).** Son ventas válidas con atributos de cliente nulos. - Inclúyalas en los totales generales; si las excluye, las cifras no cuadran con el total de la tabla. - En desgloses por segmento, ciudad o departamento, muéstrelas como una categoría explícita (por ejemplo, "Sin CRM") en lugar de dejarlas en blanco o filtrarlas. - `cod_cliente` sí está informado en estas filas, por lo que el conteo de clientes distintos las incluye.
>
> **Departamento.** Tiene 67 nulos frente a 8 en `ciudad`: un análisis por departamento deja fuera más filas que uno por ciudad. Use una categoría "Sin departamento".
>
> **Filas descartadas.** Las 35 filas con cantidad o precio nulos no se imputan ni se conservan, así que los totales pueden ser inferiores a los del ERP.
>
> **Ubicación.** Ciudad y departamento son los del cliente según el CRM, no los del lugar de la venta.
>
> **Valores de texto.** - `Hogar` existe como segmento y como categoría; no los confunda en filtros. - `Tecnologia` está escrita sin tilde.
>
> **Moneda.** Todos los valores están en COP; la tabla no incluye conversión a otras monedas ni indica el tratamiento de impuestos (Por confirmar).

6.5 Análisis crítico

- **¿Fue útil?** Sí, con revisión. La estructura es la solicitada, las cifras de calidad son las que le di y respetó la instrucción de no inventar: marcó como "Por confirmar" el responsable, la frecuencia de actualización y la fecha de ejecución.
- **Lo que aportó más allá de lo pedido.** Las advertencias de uso son valiosas y correctas: que `Hogar` existe a la vez como segmento y como categoría, que las devoluciones distorsionan el ticket promedio y que la ciudad es la del cliente y no la del punto de venta.
- **Ajustes necesarios tras mi revisión:**
  1. Afirma que la unicidad de `id_venta` no está validada. Sí lo está (el ETL tiene un `assert`), pero yo no lo dije en el prompt. Es un error por omisión mía, no del modelo.
  2. Los ejemplos del diccionario son coherentes pero inventados: "Comercial Andina S.A.S." no existe en los datos y "Silla ergonómica" aparece con tilde, mientras que en el catálogo está sin tilde. Antes de publicar el documento hay que reemplazarlos por valores reales.
  3. Marcó como "por confirmar" el origen de `nombre_producto`, `categoria` y `canal`, porque el prompt no indicaba qué columna viene de qué archivo. La cautela es correcta; el vacío era mío.
- **Impacto en la tarea.** Redactar este documento a mano toma más de una hora. Con el prompt refinado se obtiene un borrador sólido en minutos, pero la revisión humana sigue siendo obligatoria: en documentación, un ejemplo inventado es un dato falso que alguien puede tomar por cierto.

---

7. Prompt 4 — Modelo dimensional (esquema en estrella)

Tarea: convertir la tabla plana en un esquema en estrella para Power BI. Este prompt se formuló directamente en su versión estructurada, aplicando lo aprendido en los tres anteriores.

7.1 Prompt

```
Actúa como arquitecto de datos especializado en modelado dimensional (Kimball) y DuckDB.

CONTEXTO
Tengo una tabla plana `ventas_integradas.parquet` (una fila por venta, año 2025, moneda COP) y necesito
convertirla en un esquema en estrella para consumirlo desde Power BI en modo importación. Columnas y tipos de la
tabla plana:

| Columna | Tipo | Nota |
|---|---|---|
| id_venta | VARCHAR | llave única, ej. V00296 |
| fecha | DATE | 2025-01-01 a 2025-12-31 |
| cod_cliente | VARCHAR | ej. C0119 |
| razon_social, segmento, ciudad | VARCHAR | nulos cuando `cliente_sin_crm` es true |
| departamento | VARCHAR | además puede venir nulo aunque el cliente exista |
| sku | VARCHAR | ej. SKU-026 |
| nombre_producto, categoria | VARCHAR | |
| canal | VARCHAR | Tienda, Web, Televentas |
| cantidad | BIGINT | negativa en devoluciones |
| precio_unitario, costo_unitario, total_venta, margen | DOUBLE | |
| es_devolucion, cliente_sin_crm | BOOLEAN | |

TAREA
Escribe un único script SQL para DuckDB que lea `ventas_integradas.parquet` y genere cuatro archivos Parquet:

1. `dim_fecha.parquet`: un registro por cada día del 2025-01-01 al 2025-12-31 (calendario completo, aunque un día
no tenga ventas). Columnas: `fecha` (DATE, llave), `anio`, `trimestre` (texto `T1`..`T4`), `mes_num`,
`mes_nombre` (en español, ej. `Enero`), `anio_mes` (texto `2025-01`), `dia_semana_num` (1 = lunes ... 7 =
domingo), `dia_semana_nombre` (en español), `es_fin_de_semana` (BOOLEAN).
2. `dim_cliente.parquet`: un registro por `cod_cliente`. Columnas: `cod_cliente` (llave), `razon_social`,
`segmento`, `ciudad`, `departamento`. Los atributos nulos se reemplazan por el texto `Sin información`, de modo
que los clientes sin CRM también tengan su fila y no se pierdan ventas al relacionar.
3. `dim_producto.parquet`: un registro por `sku`. Columnas: `sku` (llave), `nombre_producto`, `categoria`,
`costo_unitario`.
4. `fact_ventas.parquet`: una fila por venta. Columnas: `id_venta`, `fecha`, `cod_cliente`, `sku`, `canal`,
`cantidad`, `precio_unitario`, `total_venta`, `margen`, `es_devolucion`, `cliente_sin_crm`.

REGLAS
- Los nombres de mes y de día deben quedar en español sin depender de la configuración regional del sistema (usa
`CASE` o una lista, no `strftime('%B')`).
- Usa `COPY (...) TO '...' (FORMAT PARQUET)`.
- Al final incluye consultas de validación que devuelvan una fila cada una con columnas `validacion`, `resultado`
(`OK` o `ERROR`) y `detalle`: (a) las llaves de cada dimensión son únicas; (b) no hay llaves huérfanas de la
tabla de hechos en ninguna dimensión; (c) el número de filas y la suma de `total_venta` de `fact_ventas`
coinciden con los de la tabla plana; (d) `dim_fecha` tiene 365 filas.

FORMATO DE LA RESPUESTA
- Un único bloque de código SQL, con comentarios breves en español, ejecutable de principio a fin con DuckDB 1.x.
- Después del código, máximo 4 viñetas con decisiones de diseño o supuestos. Sin más explicaciones.
```

7.2 Resultado generado por la IA (literal)

> ```
> -- ======================================================================
> -- Esquema en estrella a partir de ventas_integradas.parquet (DuckDB 1.x)
> -- ======================================================================
>
> -- 0) Carga de la tabla plana en una tabla temporal
> CREATE OR REPLACE TEMP TABLE ventas AS
> SELECT * FROM read_parquet('ventas_integradas.parquet');
> ```

```
-- 1) dim_fecha: calendario completo de 2025 (un registro por día)
COPY (
    WITH calendario AS (
        SELECT CAST(d AS DATE) AS fecha
        FROM generate_series(DATE '2025-01-01', DATE '2025-12-31', INTERVAL 1 DAY) AS t(d)
    )
    SELECT
        fecha,
        CAST(year(fecha) AS INTEGER)                    AS anio,
        concat('T', quarter(fecha))                     AS trimestre,
        CAST(month(fecha) AS INTEGER)                   AS mes_num,
        -- Nombres en español por lista (índice base 1), sin depender del locale
        ['Enero', 'Febrero', 'Marzo', 'Abril', 'Mayo', 'Junio', 'Julio', 'Agosto',
         'Septiembre', 'Octubre', 'Noviembre', 'Diciembre'][month(fecha)] AS mes_nombre,
        strftime(fecha, '%Y-%m')                        AS anio_mes,
        CAST(isodow(fecha) AS INTEGER)                  AS dia_semana_num,   -- 1 = lunes ... 7 = domingo
        ['Lunes', 'Martes', 'Miércoles', 'Jueves', 'Viernes', 'Sábado',
         'Domingo'][isodow(fecha)]                      AS dia_semana_nombre,
        isodow(fecha) >= 6                              AS es_fin_de_semana
    FROM calendario
    ORDER BY fecha
) TO 'dim_fecha.parquet' (FORMAT PARQUET);

-- 2) dim_cliente: un registro por cod_cliente; nulos -> 'Sin información'
COPY (
    SELECT
        cod_cliente,
        -- MAX ignora nulos: toma el valor informado si existe en alguna venta
        COALESCE(MAX(razon_social), 'Sin información') AS razon_social,
        COALESCE(MAX(segmento),     'Sin información') AS segmento,
        COALESCE(MAX(ciudad),       'Sin información') AS ciudad,
        COALESCE(MAX(departamento), 'Sin información') AS departamento
    FROM ventas
    GROUP BY cod_cliente
    ORDER BY cod_cliente
) TO 'dim_cliente.parquet' (FORMAT PARQUET);

-- 3) dim_producto: un registro por sku (atributos de la venta más reciente)
COPY (
    SELECT sku, nombre_producto, categoria, costo_unitario
    FROM (
        SELECT
            sku, nombre_producto, categoria, costo_unitario,
            ROW_NUMBER() OVER (PARTITION BY sku ORDER BY fecha DESC, id_venta DESC) AS rn
        FROM ventas
    )
    WHERE rn = 1
    ORDER BY sku
) TO 'dim_producto.parquet' (FORMAT PARQUET);

-- 4) fact_ventas: una fila por venta, solo llaves, medidas e indicadores
COPY (
    SELECT
        id_venta,
        CAST(fecha AS DATE) AS fecha,
        cod_cliente,
        sku,
        canal,
        cantidad,
        precio_unitario,
        total_venta,
        margen,
        es_devolucion,
        cliente_sin_crm
    FROM ventas
    ORDER BY fecha, id_venta
) TO 'fact_ventas.parquet' (FORMAT PARQUET);

-- ======================================================================
-- VALIDACIONES (se leen los Parquet generados; una fila por validación)
-- ======================================================================
```

```
-- (a) Llaves únicas en cada dimensión
SELECT 'a1. Llave única en dim_fecha' AS validacion,
       CASE WHEN count(*) = count(DISTINCT fecha) THEN 'OK' ELSE 'ERROR' END AS resultado,
       concat('filas=', count(*), ' | llaves distintas=', count(DISTINCT fecha)) AS detalle
FROM read_parquet('dim_fecha.parquet')

UNION ALL
SELECT 'a2. Llave única en dim_cliente',
       CASE WHEN count(*) = count(DISTINCT cod_cliente) THEN 'OK' ELSE 'ERROR' END,
       concat('filas=', count(*), ' | llaves distintas=', count(DISTINCT cod_cliente))
FROM read_parquet('dim_cliente.parquet')

UNION ALL
SELECT 'a3. Llave única en dim_producto',
       CASE WHEN count(*) = count(DISTINCT sku) THEN 'OK' ELSE 'ERROR' END,
       concat('filas=', count(*), ' | llaves distintas=', count(DISTINCT sku))
FROM read_parquet('dim_producto.parquet')

-- (b) Sin llaves huérfanas de la tabla de hechos
UNION ALL
SELECT 'b1. Huérfanas fact -> dim_fecha',
       CASE WHEN count(*) = 0 THEN 'OK' ELSE 'ERROR' END,
       concat('filas huérfanas=', count(*))
FROM read_parquet('fact_ventas.parquet') f
LEFT JOIN read_parquet('dim_fecha.parquet') d ON f.fecha = d.fecha
WHERE d.fecha IS NULL

UNION ALL
SELECT 'b2. Huérfanas fact -> dim_cliente',
       CASE WHEN count(*) = 0 THEN 'OK' ELSE 'ERROR' END,
       concat('filas huérfanas=', count(*))
FROM read_parquet('fact_ventas.parquet') f
LEFT JOIN read_parquet('dim_cliente.parquet') d ON f.cod_cliente = d.cod_cliente
WHERE d.cod_cliente IS NULL

UNION ALL
SELECT 'b3. Huérfanas fact -> dim_producto',
       CASE WHEN count(*) = 0 THEN 'OK' ELSE 'ERROR' END,
       concat('filas huérfanas=', count(*))
FROM read_parquet('fact_ventas.parquet') f
LEFT JOIN read_parquet('dim_producto.parquet') d ON f.sku = d.sku
WHERE d.sku IS NULL

-- (c) Conciliación de filas y de total_venta contra la tabla plana
UNION ALL
SELECT 'c1. Número de filas fact vs tabla plana',
       CASE WHEN f.n = p.n THEN 'OK' ELSE 'ERROR' END,
       concat('fact=', f.n, ' | plana=', p.n)
FROM (SELECT count(*) AS n FROM read_parquet('fact_ventas.parquet')) f
CROSS JOIN (SELECT count(*) AS n FROM read_parquet('ventas_integradas.parquet')) p

UNION ALL
SELECT 'c2. Suma total_venta fact vs tabla plana',
       -- Tolerancia de 1 centavo por redondeo de punto flotante (DOUBLE)
       CASE WHEN abs(f.s - p.s) < 0.01 THEN 'OK' ELSE 'ERROR' END,
       concat('fact=', round(f.s, 2), ' | plana=', round(p.s, 2))
FROM (SELECT COALESCE(sum(total_venta), 0) AS s FROM read_parquet('fact_ventas.parquet')) f
CROSS JOIN (SELECT COALESCE(sum(total_venta), 0) AS s FROM read_parquet('ventas_integradas.parquet')) p

-- (d) dim_fecha con 365 filas
UNION ALL
SELECT 'd1. dim_fecha tiene 365 filas',
       CASE WHEN count(*) = 365 THEN 'OK' ELSE 'ERROR' END,
       concat('filas=', count(*))
FROM read_parquet('dim_fecha.parquet')

ORDER BY validacion;
```

- **Llaves naturales:** se usan `fecha`, `cod_cliente` y `sku` como llaves de relación (sin llaves subrogadas), suficiente para Power BI en modo importación con un solo año y sin historial de cambios (SCD tipo 1). `canal` queda como dimensión degenerada en la tabla de hechos.

- **Atributos inconsistentes:** si un cliente trae un atributo nulo en unas ventas e informado en otras, se conserva el informado (`MAX` ignora nulos); si un `sku` tiene varios nombres, categorías o costos, se toma el de la venta más reciente, por lo que `dim_producto.costo_unitario` es el costo vigente y no el histórico de cada venta.
- **Supuesto de llaves no nulas:** se asume que `cod_cliente`, `sku` y `fecha` nunca son nulos y que todas las fechas caen en 2025; si no se cumple, las validaciones `a` y `b` lo reportan como `ERROR`.
- **Validaciones:** se unen con `UNION ALL` para ver las nueve filas en un solo resultado (cada consulta aporta exactamente una fila), y la suma de `total_venta` se compara con tolerancia de 0,01 por ser `DOUBLE`.

7.3 Ejecución real

Salida de las consultas de validación incluidas en el propio script:

```
a1. Llave única en dim_fecha | OK | filas=365 | llaves distintas=365
a2. Llave única en dim_cliente | OK | filas=127 | llaves distintas=127
a3. Llave única en dim_producto | OK | filas=32 | llaves distintas=32
b1. Huérfanas fact -> dim_fecha | OK | filas huérfanas=0
b2. Huérfanas fact -> dim_cliente | OK | filas huérfanas=0
b3. Huérfanas fact -> dim_producto | OK | filas huérfanas=0
c1. Número de filas fact vs tabla plana | OK | fact=1465 | plana=1465
c2. Suma total_venta fact vs tabla plana | OK | fact=730737515.9 | plana=730737515.9
d1. dim_fecha tiene 365 filas | OK | filas=365
```

7.4 Análisis crítico

- **¿Fue útil?** Sí. El script corrió completo y las nueve validaciones dieron `OK`: llaves únicas, cero llaves huérfanas, mismo número de filas (1.465) y misma suma de ventas que la tabla plana, y calendario de 365 días. Las cuatro tablas resultantes son la fuente del tablero de la evidencia AA2-EV03.
- **Detalle verificado.** `dim_cliente` tiene 127 filas: los 120 clientes del CRM más 7 códigos que no existen en el CRM (sus 8 ventas se conservan con "Sin información"), tal como se pidió.
- **Pedir las validaciones dentro del resultado fue la mejor decisión del prompt.** Convierte la respuesta en algo que se demuestra solo, en lugar de algo en lo que hay que confiar.
- **Limitaciones que el modelo declaró y que comparto:** las dimensiones se derivan de la tabla de ventas, de modo que un cliente o producto sin ventas no aparecería; y `costo_unitario` es el de la venta más reciente, no un histórico. Para este caso es suficiente; en producción las dimensiones deberían venir de sus sistemas de origen.
- **Ajuste aplicado aguas abajo.** Los importes quedan como `DOUBLE`; en Power BI los convertí a decimal fijo para evitar errores de redondeo en las sumas.

---

8. Comparación entre prompt básico y prompt refinado

	Prompt básico	Prompt refinado
Extensión	1 línea	30 a 45 líneas
Qué hace el modelo con lo que no sabe	Supone (o, como en el Prompt 3, se niega a inventar)	No necesita suponer: el contexto está en el prompt
¿Corre sobre los datos reales sin tocarlo?	No (Prompts 1 y 2)	Sí, los tres resultados con código
Riesgo principal observado	Resultado equivocado sin error visible: 0 de 1.525 filas	Supuestos declarados que hay que revisar
Decisiones de negocio	Las toma el modelo (descartar devoluciones)	Las toma el analista, por escrito
Verificación	Hay que construirla aparte	Viene incluida (`assert`, reporte, validaciones SQL)

9. Consideraciones éticas y de uso responsable

- **Datos.** Todo el ejercicio usa datos simulados. En un caso real no se deben pegar registros con datos personales en un prompt: se usa una muestra anonimizada o solo el esquema, en línea con los principios de minimización y finalidad de la Ley 1581 de 2012.
- **Responsabilidad.** El modelo genera; quien firma el resultado soy yo. Por eso cada salida se ejecutó y se contrastó con una segunda implementación antes de aceptarla.
- **Transparencia.** Este informe declara qué herramienta se usó, qué generó la IA (se reproduce literal) y qué es análisis propio.
- **Veracidad.** La instrucción "no inventes; escribe *Por confirmar*" redujo las afirmaciones sin sustento, pero no las eliminó: los ejemplos inventados del Prompt 3 lo demuestran.

10. Conclusiones

1. **La calidad del resultado depende más del prompt que del modelo.** Con la misma herramienta, una frase produjo un script que descartaba todos los datos y un prompt estructurado produjo un proceso correcto al primer intento.
2. **Un buen prompt de integración de datos se parece a una buena especificación técnica:** fuentes con muestra real, problemas conocidos, reglas de negocio numeradas, esquema de salida exacto y criterios de validación. Si no puedo escribir eso, todavía no entiendo el problema lo suficiente como para delegarlo.
3. **Pedir la verificación como parte de la respuesta** (`assert`, reportes de conteo, consultas de validación) es la técnica que más valor aportó, porque hace comprobable el resultado.
4. **El error más peligroso no es el que detiene el programa, sino el que termina sin avisar.** El caso de las 0 filas muestra por qué en integración de datos siempre hay que conciliar conteos de entrada y salida.
5. **La IA también revisa al analista.** Detectó una contradicción en mi requerimiento (lectura *lazy* con latin-1) y se negó a documentar una tabla que no le había mostrado.
6. **Aplicabilidad real.** Los cuatro prompts son reutilizables en proyectos reales cambiando la muestra y las reglas: aplanar exportaciones de un CRM, integrar un ERP con catálogos, documentar tablas y

generar modelos dimensionales son tareas cotidianas en ingeniería de datos. El tiempo ahorrado es grande, siempre que una parte se reinvierta en verificar.

11. Referencias

Anthropic. (2026). *Claude* [Modelo de lenguaje de gran tamaño]. https://claude.ai

DuckDB Foundation. (2026). *DuckDB documentation*. https://duckdb.org/docs/

Polars. (2026). *Polars user guide*. https://docs.pola.rs/

Servicio Nacional de Aprendizaje [SENA]. (2025). *Estrategias de integración y análisis de datos mediante inteligencia artificial* [Componente formativo]. Programa Aplicación de la inteligencia artificial en la integración de datos.

Servicio Nacional de Aprendizaje [SENA]. (2025). *Recolección, tratamiento y visualización de datos para inteligencia artificial* [Componente formativo]. Programa Aplicación de la inteligencia artificial en la integración de datos.

Anexo. Cómo reproducir los resultados

```
python datos/generar_datos.py                          # genera las tres fuentes simuladas
python AA2-EV01-informe-prompts/ejecutar_pipeline.py   # extrae el código de las respuestas y lo ejecuta
```

Los prompts están en `prompts/`, las respuestas literales del modelo en `respuestas/`, el código extraído en `scripts/` y la salida de consola en `ejecucion.log`.